Cyber Data Acquisition / Synthetic Data Generation Report

Purpose:
This report details how the research data was obtained or generated, its characteristics, quality controls, and any preprocessing or transformations performed prior to experimentation. It ensures the data used in analysis is **transparent**, **ethical**, and **replicable**.

---

1. Project Context

Field	Response
Student Name(s)	Denzel Jefferson
Research Title	Contextualized Threat Intelligence Prioritization for CTI Analysts
Faculty Advisor	Professor Julie Henderson
DoD/ACE Mentor	Professor William Littleton
Version	v2.0
Date	May 4, 2026

---

2. Purpose of Data in Research

Describe how the data supports the research question and hypothesis.

Include:

- What is the **role of the data** (e.g., training, testing, validation, simulation)?
- How does the data relate to the **independent/dependent variables** in your study?
- What is the **expected outcome** from using this dataset?

Example:

The dataset will be used to train an anomaly detection model to identify backdoor poisoning attempts in a simulated IDS environment.

The dataset is used to simulate operational and threat intelligence events to evaluate how contextual environmental data from Elasticsearch can improve automated threat prioritization in MISP. The data serves as the primary input for both the automated and

manual prioritization processes, and the analysis compares the two methods. In this research, the dataset serves as both test data and simulation data. The dataset directly relates to the independent and dependent variables. The independent variable is the method of threat prioritization. In contrast, the dependent variables include the time required to identify high-priority threats, threat prioritization accuracy, and alert fatigue, measured through improvements in signal-to-noise ratio and reductions in alert volume.

The expected outcome of this dataset is to demonstrate that including contextual environmental data in automated prioritization workflows will reduce overall analyst workload, improve prioritization accuracy, and decrease the time required to identify critical threats.

---

3. Data Type and Source

Data Type	Description	Source / Provider	Access Method	Licensing / Ethics
Synthetic operational logs	Simulated system alerts, events, and analyst workflow data	Python scripts	Local directory	Internal research use, no real user data
Simulated threat intelligence alerts using rule-based contextual scoring	Automated threat indicators and alert events for CTI workflows	Log Generator (Docker Container)	Local Docker container	Internal synthetic datasets, no licensing
Simulated contextual environmental data	Environmental data used to assess prioritization efficiency	Python-generated data	Script based	Internal research use
Elasticsearch datasets	Storing synthetic logs and contextual metadata for further analysis	Elasticsearch platform	Filebeat	Local research environment, isolated network

Note: Examples included in above table must be replaced with your data.

Include:

- Whether data is **real-world**, **synthetic**, **simulated**, or **hybrid**
- **Origin of the data** (organization, publication, or tool)
- **Access method** (API, file download, generator script)
- **Data sharing/license terms**
- **IRB or ethical considerations** (if applicable)

Tip: If synthetic data was generated, explain **why real data was unavailable** or **why simulation was chosen**.

The data used in this research is synthetic and simulated, generated within a controlled testbed environment. Real-world data was not used due to ethical concerns, security, and DoD restrictions. Synthetic data ensures that no personal, classified, or operational data is left exposed. Synthetic operational logs and threat intelligence alerts were generated using Python scripts and a Docker container log generator. Contextual environmental data is also simulated to analyze how contextual information affects automated versus manual threat prioritization decisions. All data is generated locally within the isolated testbed environment and accessed via Filebeat. No external datasets were used as the dataset is intended for academic and research purposes.

---

4. Data Acquisition / Generation Process

Describe **how** data was collected, captured, or generated.

Include:

- Hardware or software tools used
- Collection frequency (intervals, duration)
- Variables collected (features, labels, timestamps)
- Scripts or algorithms used for data generation

Step	Process Description	Tool / Script	Duration / Frequency	Output
1	Generate synthetic operational system logs	Python log generation scripts	1 run per 10 min	Indexed log entries in Elasticsearch

Cyber Data Acquisition / Synthetic Data Generation Report

Step	Process Description	Tool / Script	Duration / Frequency	Output
2	Generate simulated CTI alerts and threats	Log Generator (Docker Container)	Continuous through the experiment window	Simulated alert logs
3	Generate contextual environmental metadata	Python contextual data script	Once per experiment run	Contextual metadata
4	Ingest logs and metadata into Elasticsearch	Filebeat	Real-time ingestion	Elasticsearch datasets
5	Applies weighted contextual scoring logic to classify alerts (High, Medium, Low)	Simulated MISP-based scoring logic	During the experiment window	Threat prioritization scores
6	Collect experiment results	Python experiment script	End of each experiment run	Elasticsearch datasets

Note: Examples included in above table must be replaced with your data.

Example:

Data was generated using a custom Python script simulating user login attempts and system alerts at random intervals to represent network noise and malicious activity.

Data for this research experiment was generated using custom Python scripts and Docker container Log generation tools within the isolated testbed environment. The data generation process is designed to simulate realistic CTI analyst workflows, including operational system movement and threat intelligence events.

Operational logs were generated using Python scripts that simulate normal system operations and threat-related events. Simulated threat intelligence alerts were generated using a Docker container that emulates high-alert environments. Contextual environmental data was generated to represent the clutter that can influence prioritization decisions.

Filebeat was used to ingest generated logs and metadata into Elasticsearch. MISP APIs were used to produce prioritization scores and process threat intelligence events. A Python script controlled each experiment run, triggering log generation, collecting prioritization outputs, and storing experimental metadata for analysis.

Cyber Data Acquisition / Synthetic Data Generation Report

5. Data Characteristics

Provide a **high-level summary** of dataset structure and properties.

Attribute	Description
Data format	Structured log data stored in Elasticsearch
Number of records	TC-001: 1,000 TC-002: 1065 TC-003: 2000
Number of features	10-20 fields per log entry
Labeled/Unlabeled	Labeled- Alerts in a category of High, Medium, Low, based on a weighted contextual scoring model
Time range covered	10-minute window per experiment run
Sampling frequency	Events generated at random second intervals
Data size	5-50 MB per experiment run
Storage location	Elasticsearch stores the data in the Elastic VM and local storage directory

Note: Examples included in above table must be replaced with your data.

Include:

- Schema or field names (example snippet)
    1. Time_stamp
    2. Event_type
    3. Threat_prioritization
    4. MISP_score
    5. Host_name
    6. Threat_category
    7. Alert_ranking
- Labeling methods (manual, automated, semi-supervised)
- Examples of data entries (with sensitive data removed)

  The dataset is primarily composed of synthetic log data generated during each experiment run. Each experiment run produces between 1,000 and 2,000 log events

depending on the test case. Logs include multiple factors, such as timestamps, threat rankings, certain types, and contextual metadata, all used for prioritization. Data is labeled through MISP simulated prioritization scores and classification scores of High, Medium, and Low. Data is stored in Elasticsearch datasets for real-time visualization, analysis, and processing.

---

6. Pre-Processing Steps

Describe all **transformations, cleaning, and feature engineering** performed before data use.

Include:

- Normalization/scaling
- Data balancing (e.g., SMOTE)
- Feature selection/extraction
- Noise removal or outlier handling
- Data partitioning (train/test/validation)

The dataset was filtered before ingestion into Elasticsearch and MISP. Duplicate logs were removed, event timing was evaluated for consistency, and threat labels were classified to ensure consistency throughout the experiment.

Step	Operation	Tool / Library	Purpose	Output
1	Remove duplicate log entries	Python(Pandas)	Filter repeating events	Filtered log dataset
2	Evaluate log timestamps	Python	Ensure accurate timeline	Time dataset
3	Filter field name for consistency	Python script	Dataset prepared for Elasticsearch ingestion and scoring classification processing	Standardized schema naming
4	Convert threat labels into classification	Python	Prioritization comparison	Labeled dataset

Step	Operation	Tool / Library	Purpose	Output
	scores (High, Medium, Low)			
5	Dataset for experiment comparison (automated vs manual)	Python script	Support experiment findings	Evaluate automated and manual datasets

Note: Examples included in above table must be replaced with your data.

Note: Include sample code snippets or references to notebooks/scripts.

---

7. Data Quality & Validation

Document how data accuracy and consistency were verified.

Include:

- Validation metrics (missing values, duplicates, anomalies)
- Ground-truth checks
- Random sampling review
- Bias analysis (if applicable)

Example:

Ran completeness check: <1% missing values. Verified timestamps are sequential and within expected ranges.

Data checks were conducted to ensure the dataset is accurate. Automated Python scripts were used to detect missing entries, missing values, and inconsistent findings. Less than 1% of log entries contained missing values, and less than 0.5% had duplicate records. Timestamps were validated to ensure chronological order and confirm that all threat events fell within the simulated experimental time window. The generated alert labels were verified to align with the prioritization rankings for High, Medium, and Low. Class distribution analysis confirmed an imbalance in alerts, with a realistic simulated ratio of lower- to higher-priority alerts.

Cyber Data Acquisition / Synthetic Data Generation Report

Metric	Description	Result
Missing Values	% of null entries or empty fields	< 1%
Duplicate Records	% duplicates or repeated log entries	< 0.5%
Class Imbalance	Ratio of High to Medium to Low alerts	2:4:6
Timestamp consistency	Threat events in chronological order	Verified
Label consistency	High/Medium/Low labels	Verified

Note: Examples included in above table must be replaced with your data.

---

8. Ethical and Security Considerations

Include:

- Data anonymization steps (e.g., hashing, tokenization)
- Compliance with **DoD, FERPA, or IRB** standards
- Confirmation of **synthetic or anonymized** data (no PII/CUI)
- Secure storage and access control measures

Example:

All personally identifiable information was replaced with synthetic identifiers. Data stored on encrypted ACE lab drive, access limited to research team.

All data used in this research experiment is synthetic and contains no classified information, PII, or CUI. Simulated data is anonymized through random generation and tokenization. The project complies with the DoD and university research guidelines, and no IRB is required. Data is stored safely in an isolated simulated environment with restricted access.

---

9. Data Sampling & Collection Intervals

Describe when and how often data was captured or simulated.

Parameter	Description
Sampling frequency	Synthetic log events are generated randomly during each experiment run
Duration of collection	10 minutes per experiment run
Time window(s)	Data is collected during random experiment sessions
Number of samples	1,000 log events are generated per run
Event Timing	Event timings are being recorded at the time of event retrieval, enabling real-time analysis.

Tip: If time-series data, include temporal coverage and granularity.

---

10. Data Storage & Management

Include:

- Storage location and access path
- File naming conventions
- Backup procedures
- Repository links (Git, Jira, or shared drive)
- Data versioning strategy

Example:

All raw and processed data stored under /datasets/v1.0/IDS/ with change logs in GitLab. Each experiment references specific dataset hash (MD5) for reproducibility.

All experimental data is stored on an Elasticsearch virtual machine and exported to the local directory for analysis. Files use a structured naming convention, such as experiment_run_(date), to track each experiment run. Virtual machine snapshots and local backups are created and included before each experiment run to prevent data loss. Configuration files and experimental Python scripts are stored in the project repository on GitHub for reproducibility and version control.

---

11. Limitations and Known Issues

Include:

- Incomplete features or missing labels
- Limited representativeness (e.g., small dataset size)
- Known noise or bias sources
- Synthetic realism limitations

Example:

Synthetic user activity patterns may not fully replicate adversarial behavior seen in production networks.

The dataset used in this research is fully synthetic and may not fully capture real-world business behavior or adversarial network behavior. The dataset size is small compared to production environments, which can limit productivity and overall data collection. Alert labels are generated using a simulated rule-based prioritization model rather than real analyst validation, which may lead to false positives. On the other hand, synthetic user and threat behavior may not fully capture the unpredictability of real-world cyber threats.

---

12. Summary Table

Section	Key Content
Data Type	Synthetic log and threat intelligence data
Source	Custom simulated testbed with Elasticsearch and MISP
Volume	1,000 log events per experiment run
Pre-processing	Normalization, labeling (High/Medium/Low)
Validation	Synthetic consistency checks
Security	Synthetic and Anonymized (no PII/CUI)
Storage	Elasticsearch VM and local directory
Use	Threat prioritization, evaluation, and automated CTI testing

Cyber Data Acquisition / Synthetic Data Generation Report

13. Mentor Review / Approval

Reviewer	Role	Date	Notes
Student Peer			
Faculty Advisor			
DoD/ACE Mentor			

Submission Guidance

Discuss with assigned ACE Team Member prior to submission of any datasets to determine expected process and necessary artifacts (e.g. scripts, notebooks, documents, etc.)

- Attach datasets or sample excerpts (if size allows).
- Include any scripts or notebooks used for generation or cleaning.
- Provide references or DOIs for any public datasets.
- Submit as **.docx** or **.pdf**, with code or data links as appendices.